

Documentación de la Base de Datos

Control de Acceso Biométrico con Arduino · Base de datos **proyecto_huella** (MySQL)

Sistema de acceso de doble factor (huella dactilar + PIN). El **sensor AS608** almacena internamente las plantillas de huella indexadas por un identificador numérico (`huella_id`, 1–127). La base de datos **no** guarda la huella: conserva el resto de los datos del usuario (nombre, RUT, PIN, rol y estado) y los vincula a ese `huella_id`. La aplicación de escritorio en Python actúa de puente entre el firmware del Arduino (por puerto serial) y MySQL.

1. Conexión

Parámetro	Valor
Motor	MySQL (probado con XAMPP / MariaDB)
Base de datos	<code>proyecto_huella</code>
Host	<code>localhost</code>
Usuario	<code>root</code>
Contraseña	(vacía)
Tablas	1 · usuarios

Nota: estas credenciales están escritas directamente en el código (`login.py`, `gestionar.py` e `ingresar.py`); no se leen del archivo `.env`. El `.env` solo guarda las credenciales maestras del administrador (`ADMIN_USER` / `ADMIN_PIN`).

2. Esquema de la tabla usuarios

Campo	Tipo	Restricciones	Descripción
<code>id</code>	<code>INT</code>	PK, AUTO_INCREMENT	Identificador interno autoincremental.
<code>huella_id</code>	<code>INT</code>	UNIQUE, NOT NULL	ID de la plantilla en el sensor (rango válido 1–127).
<code>nombre</code>	<code>VARCHAR(50)</code>	NOT NULL	Nombre del usuario que se muestra en la LCD y la interfaz.
<code>rut</code>	<code>VARCHAR(12)</code>	UNIQUE	RUT chileno con dígito verificador (formato 12345678-9).
<code>pin</code>	<code>VARCHAR(5)</code>	NOT NULL	PIN de exactamente 5 dígitos (segundo factor).
<code>rol</code>	<code>ENUM('empleado', 'admin')</code>	DEFAULT 'empleado'	Define quién puede entrar a Gestionar.
<code>estado</code>	<code>ENUM('activo', 'inactivo')</code>	DEFAULT 'activo'	Solo los usuarios 'activo' pueden ingresar.
<code>creado_en</code>	<code>TIMESTAMP</code>	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de alta del registro.

DDL — definición de la tabla

```
CREATE TABLE usuarios (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  huella_id   INT UNIQUE NOT NULL,  
  nombre      VARCHAR(50) NOT NULL,  
  rut         VARCHAR(12) UNIQUE,  
  pin         VARCHAR(5) NOT NULL,  
  rol         ENUM('empleado', 'admin') DEFAULT 'empleado',  
  estado      ENUM('activo', 'inactivo') DEFAULT 'activo',  
  creado_en   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Validaciones de la aplicación

Antes de escribir en la base de datos, `gestionar.py` valida los datos del formulario. Estas reglas complementan las restricciones de la tabla:

Campo	Regla aplicada en el código
<code>huella_id</code>	Entero entre 1 y 127, y que no exista ya en la tabla.
<code>pin</code>	Cadena de exactamente 5 dígitos.
<code>rut</code>	Número de 7 o más dígitos, guion y dígito verificador (0–9 o 'k'); además debe ser único.
<code>rol</code>	Uno de: admin, empleado.
<code>estado</code>	Uno de: activo, inactivo.

4. Consultas que ejecuta la aplicación

Autenticación de administrador — `login.py`

Valida el acceso a la sección *Gestionar* contra un administrador activo de la base de datos:

```
SELECT nombre FROM usuarios  
WHERE rut = %s AND pin = %s AND rol = 'admin' AND estado = 'activo';
```

Verificación de acceso — `ingresar.py`

Tras leer la huella, el firmware pide a la PC el nombre asociado (mensaje `CONSULTAR_ID:<id>`):

```
SELECT nombre FROM usuarios  
WHERE huella_id = %s AND estado = 'activo';
```

Luego, con el PIN tecleado (mensaje `VERIFICAR:<id>:<pin>`), se valida la combinación huella + PIN. Si hay resultado, se abre la cerradura:

```
SELECT nombre FROM usuarios  
WHERE huella_id = %s AND pin = %s AND estado = 'activo';
```

Gestión de usuarios (CRUD) — gestionar.py

```
-- Comprobaciones de unicidad
SELECT COUNT(*) FROM usuarios WHERE huella_id = %s;
SELECT COUNT(*) FROM usuarios WHERE rut = %s;

-- Alta
INSERT INTO usuarios (huella_id, nombre, rut, pin, rol, estado)
VALUES (%s, %s, %s, %s, %s, %s);

-- Listado en la tabla de la interfaz
SELECT huella_id, nombre, rut, rol, estado, creado_en
FROM usuarios ORDER BY huella_id;

-- Edición (el PIN solo se actualiza si se ingresa uno nuevo)
UPDATE usuarios SET nombre=%s, rut=%s, rol=%s, estado=%s [, pin=%s]
WHERE huella_id=%s;

-- Baja
DELETE FROM usuarios WHERE huella_id = %s;

-- Exportación a CSV
SELECT * FROM usuarios;
```

5. Cómo se usa la base de datos en cada flujo

Enrolamiento — registrar un usuario

1) El administrador elige un `huella_id` libre. 2) El sensor guarda la huella bajo ese ID. 3) Se inserta la fila en `usuarios` con nombre, RUT, PIN, rol y estado.

Verificación — abrir la puerta

1) El sensor identifica la huella y devuelve su `huella_id`. 2) La aplicación consulta el nombre del usuario **activo**. 3) El usuario teclea su PIN. 4) La aplicación confirma huella + PIN + estado activo y, si todo cuadra, responde `permitido:<nombre>` y se acciona la cerradura.

6. Notas y consideraciones

- El campo estado es el interruptor de acceso: poner a un usuario en `inactivo` le bloquea la entrada sin necesidad de borrar su huella del sensor.
- El `huella_id` es el vínculo entre el hardware (sensor) y la base de datos; debe mantenerse único y dentro del rango 1–127.
- El `pin` se almacena en texto plano (5 dígitos). Para un entorno real conviene cifrarlo o aplicarle un hash.
- El script `database/database.sql` crea solo la tabla; la base `proyecto_huella` debe existir previamente (`CREATE DATABASE proyecto_huella;`).